Автоматический мониторинг и классификация инструментов пентеста по MITRE ATT&CK

Этап 2: Демонстрация Модуля ETL

Команда проекта

Новосельский Д ▪ Коломийцев Н ▪ Толмачёв Н ▪ Ширинов А ▪ Игнатов Д

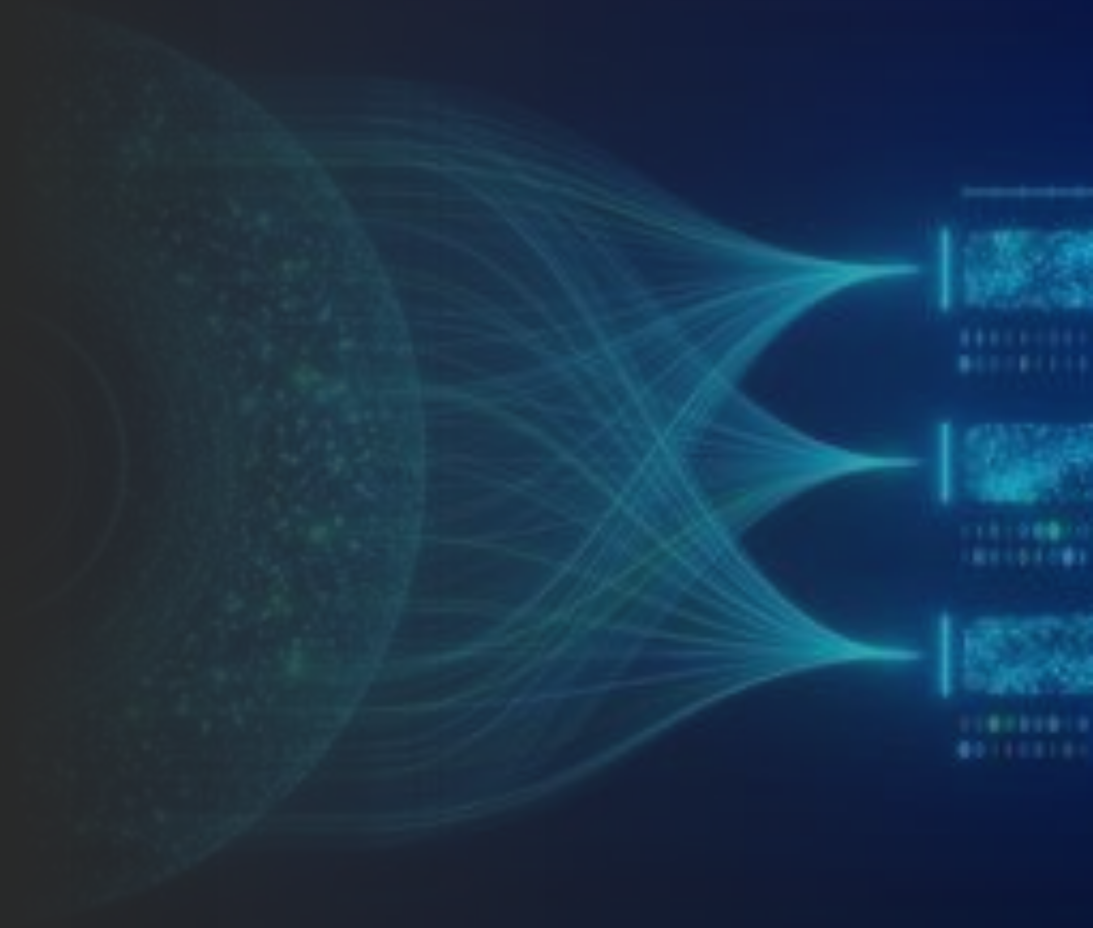
Содержание

- 01 **Архитектура ETL-пайплайна**
  Общая схема и стратегия поиска
- 02 **Extract — Сбор данных**
  GitHub API, README, raw records
- 03 **Transform — Нормализация**
  Скоринг, классификация, фильтрация
- 04 **LLM Assessment — Оценка**
  DeepSeek API, промпты, валидация
- 05 **Load — Сохранение и результаты**
  DuckDB, индексы, метрики пайплайна

01

Архитектура ETL-пайплайна

Общая схема пайплайна: от сбора данных до оценки инструментов



Что такое ETL в нашем проекте?

Extract → Transform → Load для Offensive Security Tools

EXTRACT

- GitHub API — поиск репозиториев
- PacketStorm — security tools
- RSS-фиды — новые релизы
- 150+ комбинаций запросов
- 1000+ raw records

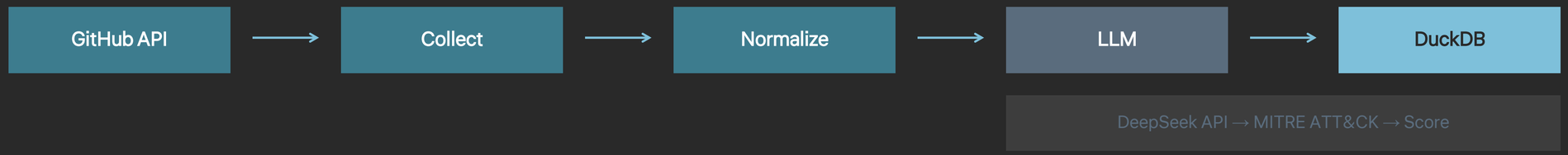
TRANSFORM

- Нормализация данных
- Pre-LLM скоринг (0.3-0.9)
- Классификация типов
- Фильтрация шума (-25%)
- 750 нормализованных

LOAD

- DuckDB — основное хранилище
- RDS — кэш для R
- JSON — интеграция
- CSV — Excel-экспорт
- Индексы для поиска

Схема потока данных



Вход: 4 источника данных × 150 запросов = 1000+ raw records

Обработка: Pre-LLM скоринг + LLM-оценка + MITRE-классификация

Выход: 600 оцененных инструментов в DuckDB с индексами

Время: Полный цикл — 28 минут

Стратегия поиска инструментов

Источники ключевых слов

- **MITRE ATT&CK** — популярные техники (TA0001, T1190...)
- **HackerOne reports** — инструменты из баг-баунти
- **GitHub trends** — растущие репозитории
- **Security conferences** — доклады и презентации
- **Reddit/Twitter** — рекомендации комьюнити

Категории ключевых слов

Reconnaissance

shodan, maltego, recon-ng, amass, subfinder

Exploitation

metasploit, burp suite, sqlmap, nuclei, exploit-db

Post-Exploitation

mimikatz, bloodhound, Empire, Cobalt Strike

Логика маппинга

MITRE TA0001 (Initial Access) → T1190 (Exploit Public-Facing App) → nuclei, burpsuite, metasploit

✓ 50+ поисковых фраз ✓ Категоризированы по типам ✓ Приоритеты high/medium/low

Целевые организации и запросы

Ключевые организации

Организация	Инструменты
projectdiscovery	Nuclei, subfinder, naabu
BishopFox	SleekyImprovement
rapid7	Metasploit Framework
BC-SECURITY	Empire, Covenant
TrustedSec	SilentTrinity
tenable	Nessus, scanners
qualys	QualysGuard, scripts

Комбинирование запросов

Шаг 1: Дефолтные запросы

"metasploit framework", "burp suite", "sqlmap"

Шаг 2: По организациям

org:rapid7, org:projectdiscovery, org:BishopFox

Шаг 3: Комбинированные

org:rapid7 "metasploit framework"

Параметры поиска

- sort=stars — сортировка по звёздам
- order=desc — сначала популярные
- per_page=100 — 100 результатов
- max_pages=10 — до 1000 результатов

Результат: 50 запросов × 3 организации = 150 комбинаций для поиска

Распределение источников:

GitHub (основной, 80% результатов) ▪ PacketStorm (15%) ▪ RSS-фиды (5%) ▪ MITRE ATT&CK (для классификации)

02

Extract

Сбор данных из источников: GitHub API, README, формирование raw records

GitHub API — поиск и извлечение

HTTP-запрос к GitHub Search API

```
GET /search/repositories
Authorization: Bearer token_here
q = "metasploit framework"
sort = "stars" & order = "desc"
per_page = 100 & max_pages = 10
```

Извлечение README.md

```
GET /repos/{owner}/{repo}/contents/README.md
# Декодируем base64 → берём первые 2000 символов
readme_preview <- substr(readme_text, 1, 2000)
```

Что извлекаем из ответа

- id — уникальный идентификатор
- name — название репозитория
- description — описание проекта
- stargazers_count — звёзды
- language — язык программирования
- topics — теги (pentest, exploit...)
- html_url — ссылка на репозиторий

Пример raw-записи

Поле	Пример	Тип
record_id	github-141432	string
name	metasploit-framework	string
stars	32 547	integer

Собрано: 1000+ raw records ▪ **Все содержат:** description + README ▪ **Источники:** GitHub, PacketStorm, RSS

03

Transform

Нормализация, скоринг и фильтрация данных перед LLM-оценкой

Pre-LLM скоринг и классификация

Логика скоринга

Условие	Score	Priority
name в seed_names	0.9	high
Ключевые слова в desc	0.7	medium
Язык: Python/Go/Ruby	+0.1	бонус
Остальное	0.3	low

Причины скоринга (reasons)

- **known_tool** — инструмент в seed-списке
- **keyword_match** — совпадение по ключевым словам
- **github_source** — источник GitHub
- **high_stars** — 1000+ звёзд
- **offensive_language** — Python/Go/Ruby

Классификация entity_type

framework → metasploit-framework, impacket

scanner → nuclei, sqlmap, nmap

exploit → exploit-db, 0day

post-exploitation → evil-winrm, mimikatz

recon → shodan, maltego, amass

Примеры расчётов

Инструмент	Score	Priority	Type
metasploit	0.9	high	framework
nuclei	0.9	high	scanner
random-tool	0.3	low	tool

Seed names: metasploit, burp, sqlmap, nuclei, impacket, bloodhound, responder, crackmapexec, evil-winrm, chisel

Фильтрация и итоговая нормализация

Фильтры отсечения шума

✗ Документация и tutorials

!grepl("documentation|tutorial|guide", desc)

✗ Непопулярные репозитории

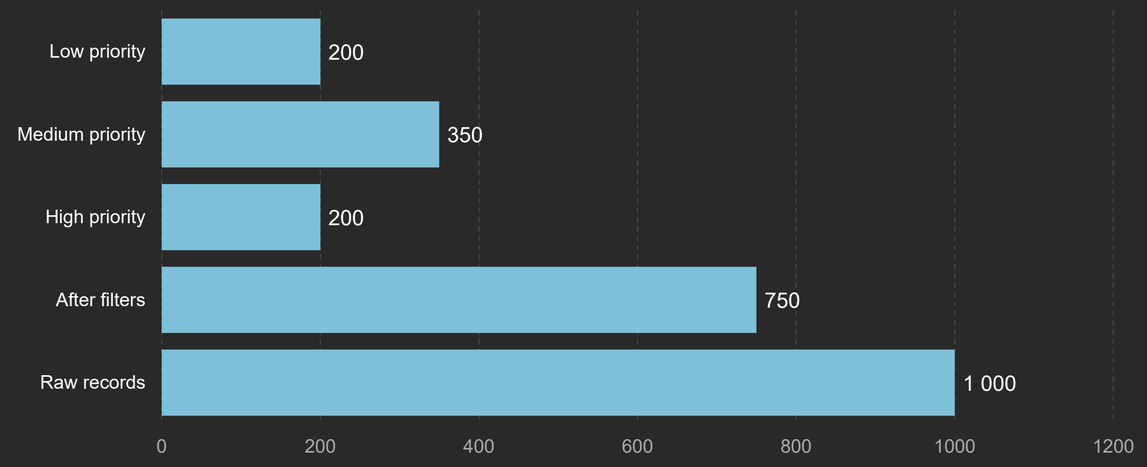
!(stars < 10 & score < 0.7)

✗ Пустой или короткий README

nchar(raw_text) > 100

✗ Неинструменты (books, courses)

Результат фильтрации



Итоговая структура записи

```
record_id = "github-141432"
name = "metasploit-framework"
source = "GitHub"
url = "https://github.com/..."
raw_description = "Metasploit Framework..."
pre_llm_score = 0.9
pre_llm_priority = "high"
entity_type = "framework"
reasons = c("known_tool", "high_stars")
```

1000 raw → 750 normalized (25% отфильтровано)

High: 200 | Medium: 350 | Low: 200

04

LLM Assessment

Оценка инструментов через DeepSeek API: промпты, валидация, MITRE-маппинг

Промпт и API DeepSeek

Структура промпта (8 вопросов)

1. Is this an OFFENSIVE security tool? (true/false)
2. Entity type: tool/framework/scanner/exploit/...
3. Summary in Russian (2-3 sentences)
4. Key capabilities in Russian (bullet points)
5. Confidence score (0.0 to 1.0)
6. Reasoning (2-3 sentences)
7. MITRE ATT&CK tactics (TA0001, TA0002...)
8. MITRE ATT&CK techniques (T1190, T1059...)

→ RETURN ONLY VALID JSON

Параметры API-запроса

POST https://api.deepseek.com/v1/chat/completions

model: "deepseek-chat"

temperature: 0.1

max_tokens: 2000 | timeout: 30s

Пример ответа JSON (Metasploit)

```
{
"is_relevant": true,
"entity_type": "framework",
"summary_ru": "Metasploit Framework — платформа для разработки и выполнения эксплойтов...",
"capabilities_ru": "- Модули сканирования уязвимостей...",
"confidence": 0.98,
"reasoning": "Очень популярный инструмент (32k звёзд)...",
"mitre_tactics": ["TA0001", "TA0002", "TA0008"],
"mitre_techniques": ["T1190", "T1059", "T1078"]
}
```

Стоимость: ~$0.01 за запрос

Валидация и Composite Score

Валидация ответа LLM

✓ Все 8 обязательных полей присутствуют

✓ confidence в диапазоне [0, 1]

✓ mitre_tactics — валидные коды (TAxxxx)

✓ mitre_techniques — валидные коды (Txxxx)

✓ is_relevant — boolean значение

Формула Composite Score

composite_score = (pre_llm_score + llm_confidence) / 2

Пример: (0.9 + 0.98) / 2 = 0.94

Финальный приоритет

≥ 0.8 → **high** (отправляем в Shiny Dashboard)

0.6 — 0.8 → **medium** (показываем в таблице)

< 0.6 → **low** (архив)

Результаты оценки

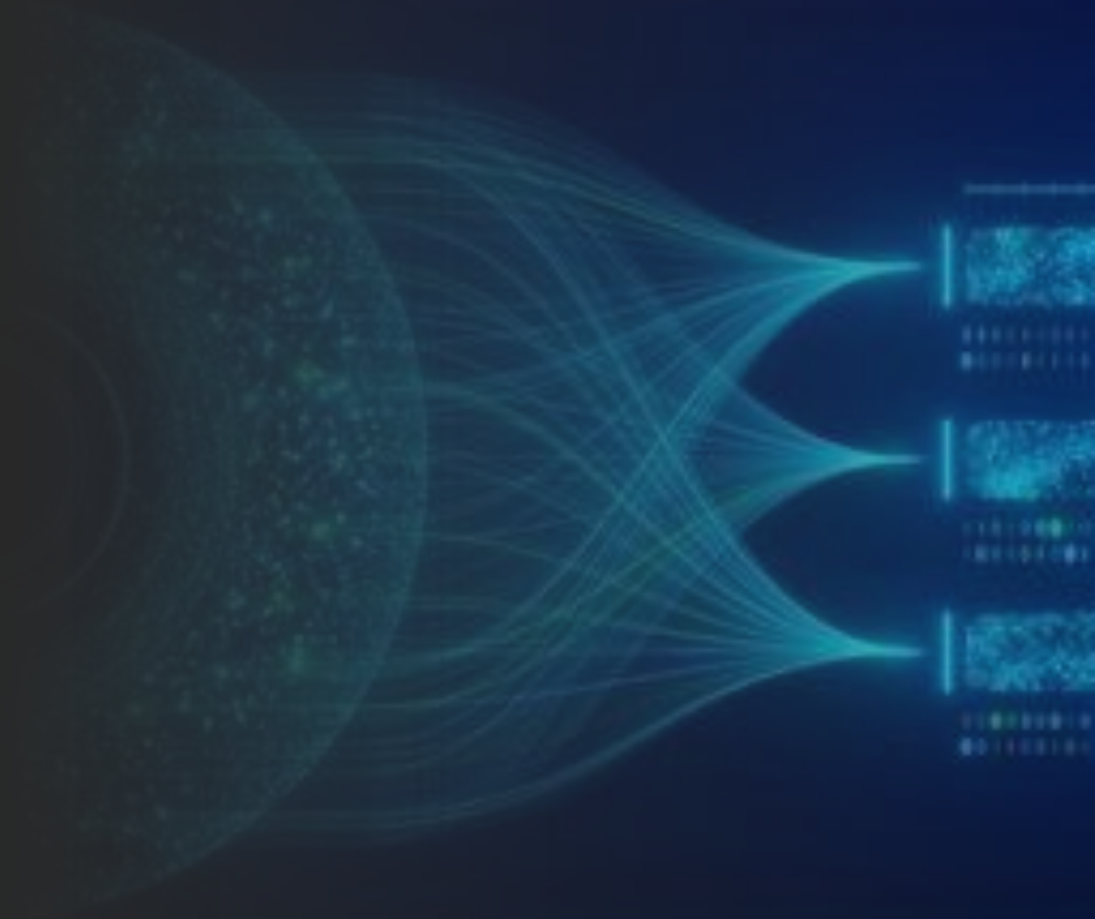
Инструмент	Pre-LLM	LLM	Composite
metasploit-fw	0.90	0.98	0.94
nuclei	0.90	0.88	0.89
burp-suite	0.90	0.94	0.92
sqlmap	0.90	0.80	0.85
bloodhound	0.90	0.82	0.86
impacket	0.70	0.86	0.78
chisel	0.50	0.86	0.68

■ High (≥0.8) ■ Medium (0.6-0.8) ■ Low (<0.6)

05

Load

Сохранение результатов в DuckDB, индексы, экспорт форматов



DuckDB, индексы и экспорт

Структура таблицы tools_assessed

```
record_id TEXT PRIMARY KEY
name TEXT
source TEXT
url TEXT
composite_score REAL
final_priority TEXT
llm_entity_type TEXT
llm_summary_ru TEXT
llm_mitre_tactics VARCHAR[]
llm_mitre_techniques VARCHAR[]
run_id TEXT
assessed_at TIMESTAMP
```

Индексы для быстрого поиска

```
CREATE INDEX idx_final_priority ON tools_assessed(final_priority)
CREATE INDEX idx_composite_score ON tools_assessed(composite_score DESC)
CREATE INDEX idx_run_id ON tools_assessed(run_id)
CREATE INDEX idx_entity_type ON tools_assessed(llm_entity_type)
```

Форматы экспорта

RDS — бинарный формат для R

saveRDS(assessed_data, "tools_assessed.rds")

JSON — для интеграции с другими системами

write_json(assessed_data, "tools_assessed.json")

CSV — для Excel и других инструментов

write.csv(assessed_data, "tools_assessed.csv")

Сравнение прогонов (run_id)

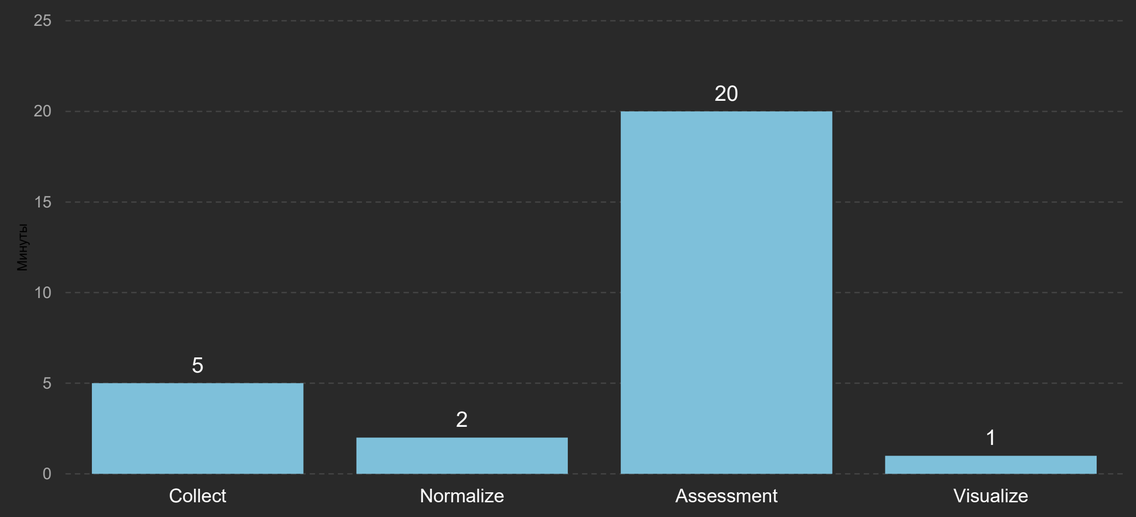
Каждый прогон получает уникальный run_id:

"20260424-143000-1234"

Позволяет отслеживать изменения между запусками

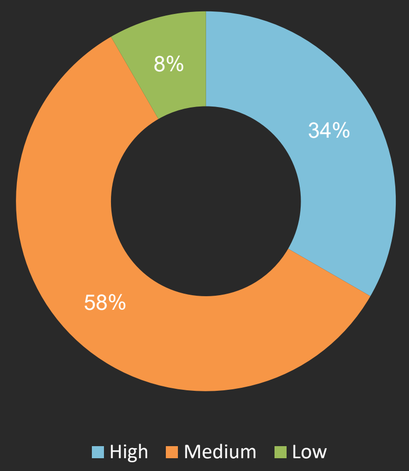
Метрики пайплайна

Время выполнения по этапам



ИТОГО: 28 минут для 600 инструментов

Распределение по приоритетам



Использование ресурсов

- API calls: 150 к GitHub
- LLM calls: 600 запросов к DeepSeek
- Disk usage: ~50 MB

Путь одного инструмента через pipeline

Пример: metasploit-framework

1. EXTRACT

Запрос:

org:rapid7

metasploit framework

Результат:

12 500 репозиториев

README:

8 000 символов

→

2. TRANSFORM

Score: 0.9

(known tool)

Priority: **high**

Entity type:

framework

Reasons:

known_tool, high_stars

→

3. LLM

Prompt:

1 542 символа

Response:

JSON с 8 полями

Confidence:

0.98

MITRE: TA0001-0008

→

4. LOAD + RESULT

Composite:

(0.9 + 0.98) / 2 = 0.94

Final priority:

high

Saved to:

tools_assessed

✓ Ready for Shiny

Ключевые особенности: Параллельная обработка 10 запросов ▪ Кэширование на каждом этапе ▪ Обработка ошибок (rate limits, broken README, corrupted JSON)

Выводы и следующие шаги

✓ ETL собирает данные из **4 источников**

✓ Pre-LLM скоринг фильтрует **25% шума**

✓ LLM добавляет **русский контекст + MITRE**

✓ Composite score объединяет оценки

✓ **28 минут** для 600 инструментов